Backend pour le suivi des élèves - Step1

Prérequis

- CLI
- modèle
- migration
- ORM

Objectif

Dans cette étape, nous allons :

- créer le projet à l'aide du kit starter
- envoyer le projet dans notre dépôt GitHub
- configurer la connexion à notre base de données MySQL
- créer la 1ère migration et le 1er modèle pour les élèves

> ⚠️ **ATTENTION**
> Avant de commencer, votre environnement de développement doit être prêt (voir l'étape 0).
> Vérifiez notamment que les conteneurs Docker de la base de données sont démarrés (`docker ps`).

Kit Starter

Nous allons commencer par créer la structure de notre projet.

Pour cela, AdonisJS propose plusieurs kits starter qui vous permettent de facilement configurer votre projet.

Nous allons passer des options à la commande pour préciser 3 choses :

- nous souhaitons réaliser une api `--kit=api`
- nous ne souhaitons pas exécuter les migrations tout de suite `--skip-migrations` (nous devons d'abord configurer MySQL)
- nous souhaitons initialiser un dépôt Git dans le projet `--git-init`

```
npm create adonisjs@latest api-suivi-eleves-etml -- --kit=api --skip-migrations --git-init
```

> 📝 **À noter**
> Le kit `api` met en place automatiquement l'authentification à l'aide de jetons d'accès (`access tokens`).
> Nous nous en préoccuperons lorsque nous mettrons en place l'authentification.

Nous obtenons une structure qui, au début, peut faire un peu peur.
Mais très vite, vous verrez que vous allez vous repérer facilement et que cela est l'assurance d'avoir un code bien organisé.

Ouvrons le dossier du projet dans VS Code :

```
cd api-suivi-eleves-etml
code .
```

Envoi du projet sur GitHub

L'option `--git-init` a initialisé un dépôt Git dans le dossier du projet, mais aucun commit n'a encore été fait.

Faisons le 1er commit, puis envoyons le projet dans le dépôt GitHub créé à l'étape 0 :

```
git add .
git commit -m "Création du projet avec le kit api d'AdonisJS"
git branch -M main
git remote add origin https://github.com/<votre-compte>/api-suivi-eleves-etml.git
git push -u origin main
```

> 📝 **À noter**
> `git branch -M main` renomme la branche courante en `main` (selon la configuration de Git, elle peut s'appeler `master`).
> `git push -u origin main` envoie la branche `main` sur GitHub et la lie à la branche distante : par la suite, un simple `git push` suffira.

> 💡 **Question**
> Le fichier `.env` contient le mot de passe de la base de données et la clé `APP_KEY`. Est-il envoyé sur GitHub ?

Non ! Le fichier `.gitignore` créé par le kit starter exclut le fichier `.env` (ainsi que le dossier `node_modules`).
Les informations sensibles restent donc sur votre PC.

Rafraîchissez la page de votre dépôt sur GitHub : vous devez y voir les fichiers du projet.

Configuration de la base de données MySQL

Par défaut, le kit `api` utilise une base de données `SQLite`.
Nous allons la remplacer par `MySQL`.

Installation du driver MySQL

Pour que l'`ORM Lucid` puisse communiquer avec `MySQL`, nous devons installer le package `mysql2` :

```
npm install mysql2
```

Les variables d'environnement

Le fichier `.env` qui contient les variables d'environnement a été créé par le kit starter.
Nous devons y ajouter, à la fin, les informations de connexion à notre base de données :

```
# Database
DB_HOST=127.0.0.1
DB_PORT=6033 # Port pour MySQL via Docker
DB_USER=root
DB_PASSWORD=root # Mot de passe pour MySQL via Docker
DB_DATABASE=db_api_eleves_step_by_step # Nom de la base de données
```

> ⚠️ **ATTENTION**
> Ne modifiez pas la variable `APP_KEY` : elle a été générée automatiquement lors de la création du projet.

AdonisJS valide les variables d'environnement au démarrage.
Nous devons donc déclarer nos nouvelles variables dans le fichier `start/env.ts` :

```
import { Env } from '@adonisjs/core/env'

export default await Env.create(new URL('../', import.meta.url), {
  // Node
  NODE_ENV: Env.schema.enum(['development', 'production', 'test'] as const),
  PORT: Env.schema.number(),
  HOST: Env.schema.string({ format: 'host' }),
  LOG_LEVEL: Env.schema.string(),

  // App
  APP_KEY: Env.schema.secret(),
  APP_URL: Env.schema.string({ format: 'url', tld: false }),

  // Session
  SESSION_DRIVER: Env.schema.enum(['cookie', 'memory', 'database'] as const),

  // Database
  DB_HOST: Env.schema.string({ format: 'host' }),
  DB_PORT: Env.schema.number(),
  DB_USER: Env.schema.string(),
  DB_PASSWORD: Env.schema.string.optional(),
  DB_DATABASE: Env.schema.string(),
})
```

La configuration de Lucid

Dans le fichier `config/database.ts`, nous remplaçons la connexion `sqlite` par la connexion `mysql` :

```
import app from '@adonisjs/core/services/app'
import env from '#start/env'
import { defineConfig } from '@adonisjs/lucid'

const dbConfig = defineConfig({
  /**
   * Connexion utilisée par défaut pour toutes les requêtes.
   */
  connection: 'mysql',

  connections: {
    /**
     * Connexion MySQL
     */
    mysql: {
      client: 'mysql2',
      connection: {
        host: env.get('DB_HOST'),
        port: env.get('DB_PORT'),
        user: env.get('DB_USER'),
        password: env.get('DB_PASSWORD'),
        database: env.get('DB_DATABASE'),
      },
      migrations: {
        naturalSort: true,
        paths: ['database/migrations'],
      },
      schemaGeneration: {
        enabled: true,
        rulesPaths: ['./database/schema_rules.js'],
      },
      debug: app.inDev,
    },
  },
})

export default dbConfig
```

> 📝 **À noter**
> Le fichier contient déjà, en commentaire, un exemple de connexion `mysql`. Il suffit de le décommenter.
> N'oubliez pas d'ajouter l'import `import env from '#start/env'` en haut du fichier.

> 📝 **À noter**
> La base de données `db_api_eleves_step_by_step` a été créée automatiquement par Docker à l'étape 0.
> Les valeurs du `.env` (port `6033`, utilisateur `root`, mot de passe `root`) correspondent à celles du fichier `docker-compose.yml`.

Création de la migration et du modèle `Student`

Nous allons partir de la base de données et commencer par les élèves.

Pour créer les fichiers de la migration et du modèle `Student`, nous allons utiliser le CLI d'AdonisJS.

Vous pouvez voir toutes les commandes qu'offre ce CLI en faisant :

```
node ace list
```

Maintenant, faisons la commande ci-dessous :

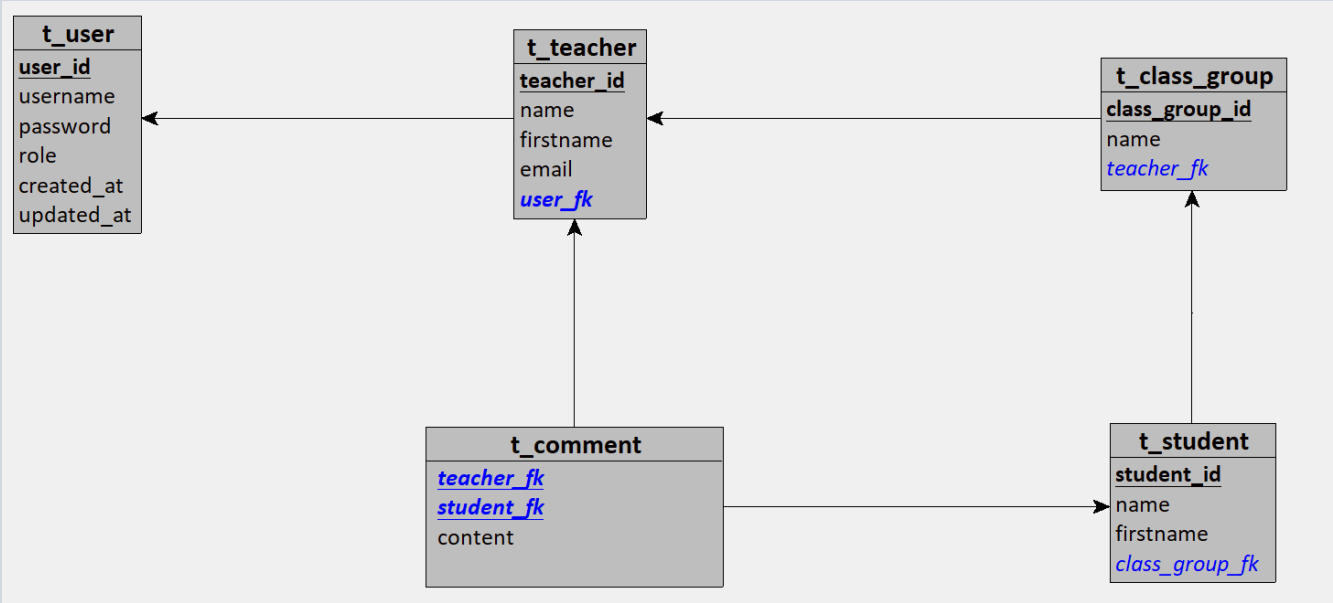
```
node ace make:model -m student
```

Cette commande crée les fichiers pour le modèle et la migration :

- `app/models/student.ts`
- `database/migrations/xxxxxxxxxxxxx_create_students_table.ts` (`xxxxxxxxxxxxx` est un timestamp)

Ces fichiers ne contiennent que la structure et c'est à vous de les compléter.

Dans un 1er temps, on ne se préoccupe pas des relations entre les tables mais seulement les champs.



La migration

Avec AdonisJS 7, **c'est la migration qui définit les champs** de notre table.

J'ajoute les champs `nom` et `prénom` à la migration :

```
import { BaseSchema } from '@adonisjs/lucid/schema'

export default class extends BaseSchema {
  protected tableName = 'students'

  async up() {
    this.schema.createTable(this.tableName, (table) => {
      table.increments('id')

      // Ajout des colonnes name et firstname
      table.string('name').notNullable()
      table.string('firstname').notNullable()

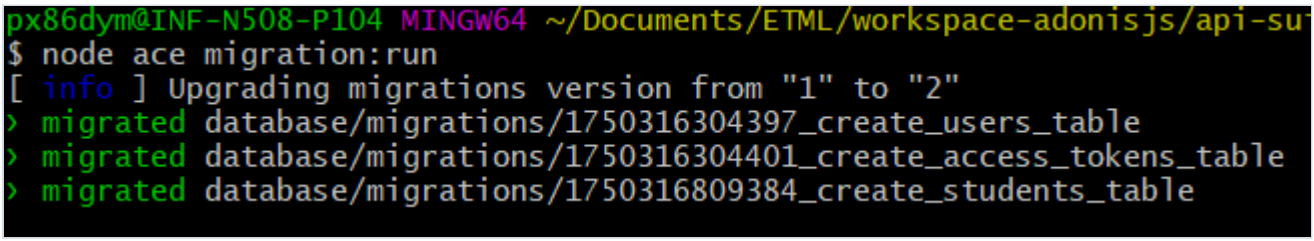
      table.timestamp('created_at')
      table.timestamp('updated_at')
    })
  }

  async down() {
    this.schema.dropTable(this.tableName)
  }
}
```

Exécution des migrations

Maintenant à l'aide du `CLI`, on peut exécuter les migrations :

```
node ace migration:run
```



> 💡 A noter:

Les migrations pour les tables `users` et `auth_access_tokens` seront exécutées également.
Nous nous en préoccuperons lorsque nous mettrons en place l'authentification.

Le schéma généré automatiquement

Après l'exécution des migrations, Lucid analyse notre base de données et génère automatiquement le fichier `database/schema.ts`.

Ce fichier contient une classe par table. Pour la table `students`, nous obtenons la classe `StudentSchema` :

```
import { BaseModel, column } from '@adonisjs/lucid/orm'
import { DateTime } from 'luxon'

export class StudentSchema extends BaseModel {
  static $columns = ['createdAt', 'firstname', 'id', 'name', 'updatedAt'] as const
  $columns = StudentSchema.$columns
  @column.dateTime({ autoCreate: true })
  declare createdAt: DateTime | null
  @column()
  declare firstname: string
  @column({ isPrimary: true })
  declare id: number
  @column()
  declare name: string
  @column.dateTime({ autoCreate: true, autoUpdate: true })
  declare updatedAt: DateTime | null
}
```

> 💡 **Question**
> Que s'est-il passé ?

Lucid a lu la structure de la table `students` et a créé automatiquement les propriétés correspondantes :

- les noms de colonnes en `snake_case` (`created_at`) deviennent des propriétés en `camelCase` (`createdAt`)
- les colonnes `nullable` ont le type `| null`
- les colonnes `timestamp` deviennent des objets `DateTime` (librairie Luxon)

> ⚠️ **ATTENTION**
> Il ne faut **jamais modifier** le fichier `database/schema.ts`.
> Il est regénéré à chaque exécution des migrations.
> Pour ajouter ou modifier un champ, il faut créer ou modifier une migration.

Le modèle

Notre modèle `Student` hérite simplement de la classe `StudentSchema`.
Il n'y a donc **rien à ajouter** pour les champs :

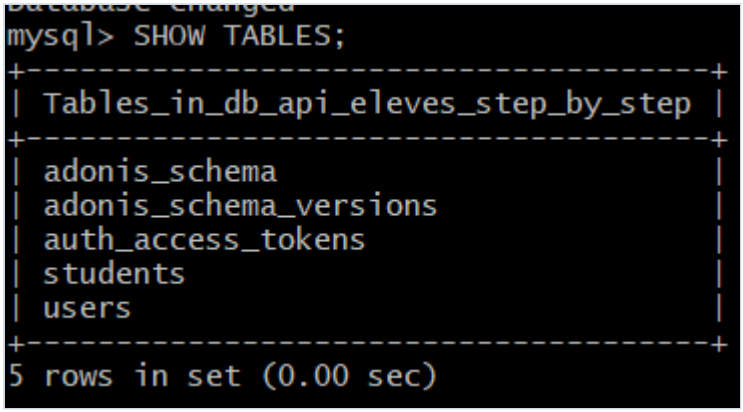
```
import { StudentSchema } from '#database/schema'

export default class Student extends StudentSchema {}
```

Le modèle servira plus tard à définir les relations avec les autres tables (classe, commentaires, ...).

Vérification de la création des tables

A vous de vous connecter à phpMyAdmin (http://localhost:8081) et de vérifier que la table `students` est bien présente.



Sauvegarde du travail

A la fin de chaque étape, pensez à faire un commit et à l'envoyer sur GitHub :

```
git add .
git commit -m "Step1 : migration et modèle Student"
git push
```